Single-Page Web-Based Notes Application

PROJECT SYNOPSIS & SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Development Team & Project Authors

AUTHOR NAME	UNIVERSITY ROLL NUMBER
ABHINAV SINGH	2308390100004
ANURAG DUBEY	2308390100014
AMAN KUMAR MAURYA	2308390100013
ADARSH GAUTAM	2308390100007
RAKESH YADAV	2308390100050

Academic Year: 2025-2026 | Department of Computer Science & Engineering

Document Generated: May 2026

Document 1: Project Synopsis Report

1. Abstract

The Single-Page Notes Application is a streamlined, client-side web application architected to allow users to efficiently create, view, and delete text-based documentation nodes. The system utilizes pure native browser mechanisms—HTML5 structural framing, CSS3 style semantics, and asynchronous client-side standard vanilla JavaScript. By avoiding cloud reliance and heavy framework abstractions, the application delivers near-zero latency interactions. Data resilience is guaranteed across device restarts and runtime session terminations via direct interaction with the browser's persistent `localStorage` API grid layout.

2. Project Objectives & Problem Definition

Modern text-capturing instruments are frequently bogged down by mandatory account authentication, external cloud latency bottlenecks, and heavy system dependency installations. This project intends to resolve these pain points by fulfilling the following milestones:

- **Minimalist Client Footprint:** Building an interface operating completely isolated from heavy external code downloads or computational server pipelines.
- **Instantaneous Transaction States:** Eliminating network-bound asynchronous load states to guarantee microsecond view performance.
- **Zero-Config Data Safety:** Implementing underlying state machines that automate non-volatile data storage safely within local browser parameters.

3. High-Level System Architecture

The application relies on a modern cohesive architecture where state tracking and view modification are managed as a single self-contained entity:

> **Architecture Framework Flow**
>
> `[User UI Interlock (DOM View)] ↔ [State Logic Loop (JavaScript Runtime)] ↔ [Hardware Sandbox Storage (Web LocalStorage)]`

When a user requests a structural update (such as an addition or erasure), the state change flows directly through the JavaScript engine, which simultaneously applies mutations across the hardware storage boundary and updates the document object layout model dynamically.

4. Critical Technical Review & Security Assessment

An internal assessment of the source components reveals specific strategic parameters regarding data handling:

- **XSS Structural Vulnerability:** The application current layout relies on assigning records via template literals bound inside `innerHTML` injection rules. If a user stores arbitrary computational script markers, the execution framework will handle it natively, exposing a vector path for cross-site script security breaches. Refactoring to `textContent` handles text arrays as safe structural nodes.
- **Garbage Management & Sanitization:** The integration of localized string manipulation mechanisms (`.trim()`) handles system filtration thresholds effectively, cutting off memory leaks resulting from empty or space-only input allocation states.

Document 2: Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This document sets out the baseline Functional and Non-Functional operational specifications governing the lifecycle deployment, automated compilation, and validation protocols for the single-page micro-notes computational utility framework.

1.2 Scope

The boundary vectors of this system are strictly confined to local client contexts. The product delivers sandboxed text storage processing, excluding synchronized collaborative workflows, database clustering, multi-user identity grouping, or dynamic rich-text parsing.

2. General System Description

2.1 Product Perspective

The application is completely self-contained and serves as an abstraction overlay mapping direct computational actions onto the browser's underlying local storage sandbox engine.

2.2 User Characteristics

The system accommodates all user types without technical prerequisites. The operational interface relies entirely on explicit button interactions, dynamic focus areas, and natural interface placeholders.

3. Functional Requirements

The underlying processes governing state mutations within the micro-app environment are categorized into four distinct functional software modules:

Module Identifier	Algorithmic Processing Sequence	System Interfaces
Module 1: Input Capture & Logic Validation	1. Intercept standard mouse triggers over UI submission pathways. 2. Fetch character values from the input field. 3. Run whitespace extraction filters using `.trim()`. 4. Validate length boundaries; throw a system warning alert block if length counts hit absolute zero.	**Input:** UI Text String **Output:** Verified Memory Allocation Block
Module 2: Persistence Synchronizer	1. Isolate the internal target state matrix array. 2. Serialize array contents into safe string objects using standard encoding metrics (`JSON.stringify`). 3. Push serialized payloads straight to the `localStorage` pipeline under specific key labels.	**Input:** Active JS Array Stack **Output:** Non-Volatile Hardware State Data
Module 3: Dynamic DOM Rendering	1. Clear active visual rows from target list components. 2. Traverse through active array datasets via execution index loops. 3. Instatitate visual component frames dynamically (`div.note`). 4. Bind operational indices straight to contextual deletion hooks.	**Input:** Encoded Key Arrays **Output:** Rendered Content Elements
Module 4: Target Record Purging	1. Track specific trigger markers identifying unique element indices. 2. Splice the data array precisely at the designated index location. 3. Propagate changes down to storage and view modules.	**Input:** Target Position Index **Output:** Mutated State Array Output

4. Non-Functional Requirements

To guarantee a professional user experience, the system conforms to the following operational parameters:

Performance Attribute	Compliance Target Parameters
Performance & Latency	Interface transformations and state computations must execute under **50 milliseconds**, keeping the experience completely lag-free.
Fault Resilience	Data must survive browser crashes, window closures, and hardware power resets. Data loss is only expected if explicit user actions purge the storage cache.
Security & Sandboxing	Enforces the standard Same-Origin Policy (SOP). Stored text data is isolated and cannot be accessed by external web domains.
Viewport Optimization	Uses fluid structural scaling optimized for views ranging from compact mobile formats up to high-resolution desktop terminals, using a max UI width restriction of **400px**.

5. Environmental Constraints & Requirements

- **Operating Software Profiles:** Platform-neutral operational capabilities. Fully operational on environments like Windows, macOS, Linux, Android, and iOS ecosystems.
- **Target Engines:** Supports web rendering environments containing standard ECMAScript engines (Chromium V8, Mozilla Gecko, Apple WebKit).
- **Hardware Allocations:** Functions comfortably on systems with 1 GHz processors and 512 MB RAM. The total storage profile requires less than 10 KB of code cache allocation space.